

RELAYSPEC: ADAPTING FROZEN SPECULATIVE DRAFTERS THROUGH LINEAR FEATURE INTERFACES

Anonymous authors

Paper under double-blind review

ABSTRACT

A speculative drafter that reads a target model’s hidden states can become poorly matched when that target changes. We study whether adaptation can remain entirely in the interface while the pretrained draft transformer stays frozen. RelaySpec learns five linear feature maps and folds them into DFlash’s existing fusion projection. Cross-model transfer reconstructs the interface expected by the drafter. Adaptation to fine-tuned targets instead learns from the target’s generated continuations using a published prefix-based token objective. Across four model transfers, three 4B specializations, two 8B specializations, two inference backends, and matched architecture and supervision studies, the method provides a useful but workload-dependent alternative to draft-body training. A 4B-trained drafter transferred to an 8B target reaches $5.82\times$ autoregressive throughput on MATH and $4.68\times$ on GSM8K, recovering 100.2% and 99.4% of native-drafter throughput. Functional interventions show why preserving the original feature path and changing context direction matter. The contribution is a tested adaptation location and an explanation of its limits, not a new denoiser, verification rule, or token loss.

1 INTRODUCTION

A fast drafter proposes several tokens, and the target language model verifies them in one parallel pass (Leviathan et al., 2023, Chen et al., 2023a). DFlash makes drafting inexpensive by predicting a masked block using intermediate target features (Chen et al., 2026). This creates an adaptation problem. A target may be replaced by a different model, or specialized with a LoRA adapter (Hu et al., 2022). Its features can then differ from those that the draft network learned to read. Updating or maintaining a separate draft transformer for every deployment is one response. We investigate changing only the feature interface.

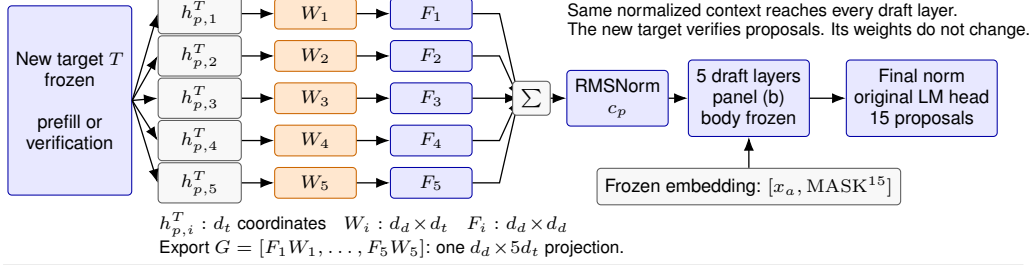
RelaySpec places five bias-free linear maps before the frozen DFlash fusion layer. Each map translates one target feature stream. Their matrix products with the original fusion are exported as a single projection. The draft transformer, normalization, embedding, and output head remain fixed. The new target still supplies context during inference. This differs from a target-independent drafter that stops using target features.

Why two training procedures? When the model changes, the interface must connect different feature spaces. We reconstruct paired features and the normalized context expected by the frozen draft body. When only a target specialization changes, identity initialization already supplies a usable interface. We then improve its token proposals with accept-until-fail training, abbreviated AUF (Yang and Li, 2026). The experiments test this distinction rather than assume either objective always wins.

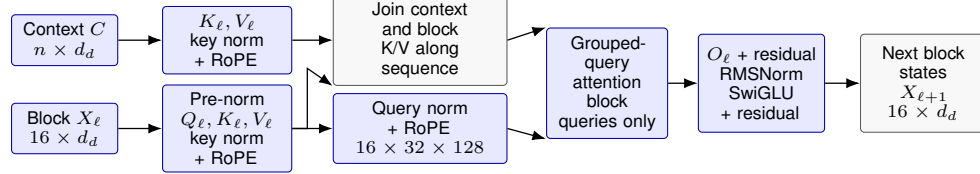
Contributions. We establish a foldable adaptation interface across model sizes, families, and target specializations. We compare seven adaptation parameterizations under two objectives and evaluate 96 supervision-budget settings. We connect the resulting speed measurements to controlled-input and live feature interventions. We also measure shared serving with native DFlash and EAGLE-3 controls. These contributions are narrower than introducing general draft adaptation (Lin et al., 2026, Ramakrishnan et al., 2025), parallel drafting (An et al., 2026a,b), or linear weight folding (Huo et al., 2026). Their value is retaining a pretrained feature-conditioned body while changing what target it can serve, with the conditions and counterexamples measured explicitly.

2 CHANGE THE INTERFACE, NOT THE DRAFT TRANSFORMER

(a) Translate five feature streams. Only the orange matrices train.



(b) Inside each frozen draft layer ℓ . Context supplies keys and values only.



Within a layer, context and block share K_ℓ, V_ℓ weights. Different layers have independent weights.

Keep projected context KV across rounds. Discard all temporary block KV before the next draft. 32 query heads, 8 KV heads, head width 128.

Figure 1: Exact adaptation location and inherited draft computation. Five maps change target features before the frozen fusion. Panel (b) expands a draft layer rather than treating it as a black box. The diagram shows the Qwen3 path used in the principal studies. Target context bypasses query projection, output projection, and the feed-forward network. Equations 1–4 specify dimensions and normalization.

2.1 FIVE FEATURE STREAMS AND ONE EXPORTED PROJECTION

Let T be the new target and R the reference model for which the fixed drafter was trained. At context position p , $h_{p,i}^T \in \mathbb{R}^{d_t}$ is target feature stream i , for $i = 1, \dots, 5$. The target width is d_t and the draft width is d_d . The original fusion consists of frozen blocks $F_i \in \mathbb{R}^{d_d \times d_d}$. We learn $W_i \in \mathbb{R}^{d_d \times d_t}$ and form the draft context

$$z_p = \sum_{i=1}^5 F_i W_i h_{p,i}^T, \quad c_p = N(z_p), \quad N(z) = g \odot \frac{z}{\sqrt{\|z\|_2^2 / d_d + 10^{-6}}}. \quad (1)$$

Here N is the inherited RMSNorm, g its frozen coordinatewise gain, and $\odot$ elementwise multiplication (Zhang and Sennrich, 2019). For same-model specialization, $W_i = I$ initially, so the initial interface is native DFlash. Cross-model maps use Xavier-uniform initialization with seed 42. They are not initialized by assuming coordinates already correspond.

Before inference, compute

$$G = [F_1 W_1, \dots, F_5 W_5] \in \mathbb{R}^{d_d \times 5d_t}, \quad c_p = N(G \text{ concat}_{i=1}^5 h_{p,i}^T). \quad (2)$$

Concatenation in Equation 2 joins feature coordinates, not token positions. Export removes a separate mapper multiplication. It does not remove storage, dispatch, or differences in projection width. Training uses $5d_t d_d$ parameters: 32.768M for 4B specialization, 83.886M for 8B specialization, and 52.429M for the Qwen3 cross-size pairs. These are full matrices, not tiny low-rank adapters.

Why linear maps? NVIDIA’s cross-model KV mapping study shows that useful internal states can sometimes be translated linearly (Heo et al., 2026). This motivates testing a linear interface, but does not establish that every pair is linearly compatible. Unlike KV transfer for skipping target prefill, RelaySpec translates hidden features for a frozen drafter. The reference language model is used only for offline supervision. We retain the new target’s own prefill and verification.

2.2 WHAT HAPPENS INSIDE THE FROZEN DRAFTER

DFlash performs one discrete masked-block prediction, not iterative diffusion in continuous space (Chen et al., 2026). At an anchor position a , the token x_a is already known. The block contains $[x_a, \text{MASK}^{15}]$. We retain **15 proposed tokens and a total block length of 16** for native and adapted DFlash. This controls proposal length, not all differences in draft capacity. The Llama reference drafter was originally trained at block length 10, which remains a declared transfer condition.

For the Qwen3-4B drafter, $d_d = 2560$, the feed-forward width is 9728, and there are five draft layers. The 8B drafter has width 4096 and feed-forward width 12288. Both have 32 query heads, 8 key/value heads, and head width 128. Thus the 4B query projection has output width 4096, not 2560. Grouped-query attention shares each KV head across four query heads (Ainslie et al., 2023). Qwen feature layers are zero-based $[1, 9, 17, 25, 33]$. The same normalized context is supplied to every draft layer.

For compact attention notation, use arrays with token positions as rows. Let $C \in \mathbb{R}^{n \times d_d}$ contain eligible context and $X_\ell \in \mathbb{R}^{16 \times d_d}$ the current block states at layer ℓ . Write $U_\ell = N_\ell^{\text{in}}(X_\ell)$ for the block pre-normalization. Each layer has its own frozen matrices $Q_\ell, K_\ell, V_\ell, O_\ell$. Its new projections are

$$Q = U_\ell Q_\ell^\top, \quad K = \text{cat}_{\text{seq}}(C K_\ell^\top, U_\ell K_\ell^\top), \quad V = \text{cat}_{\text{seq}}(C V_\ell^\top, U_\ell V_\ell^\top). \quad (3)$$

Context and block share K_ℓ, V_ℓ within a layer. The projections are independent *between* layers. After headwise query/key RMSNorm and rotary position encoding (Su et al., 2021), attention is

$$A = \text{softmax}\left(\tilde{Q} \tilde{K}^\top / \sqrt{128} + M\right) V, \quad Y = X_\ell + A O_\ell^\top, \quad X_{\ell+1} = Y + \text{MLP}_\ell(N_\ell^{\text{post}}(Y)). \quad (4)$$

The expression is evaluated per head with grouped KV sharing. M is zero for visible entries and negative infinity otherwise. The inherited MLP uses a SiLU-gated product followed by a down projection (Shazeer, 2020). Only block states undergo attention queries and MLP processing. This is the DFlash conditioning mechanism, not a new contribution of RelaySpec.

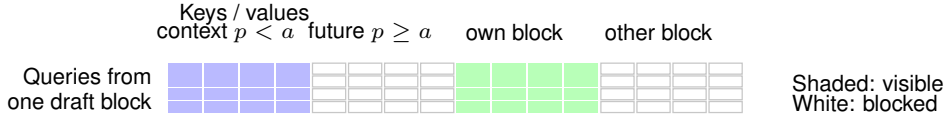


Figure 2: Training visibility for a sampled anchor. Every query in a block sees target features strictly before a and every position of its own block. It cannot see future target features or other sampled blocks. The four drawn block cells illustrate the pattern. The actual block has 16 positions.

Cache maintenance. Projected target-context KVs persist independently in each draft layer. Newly verified context is appended once. The current anchor and masked block create temporary KVs. Immediately after drafting, those temporary block entries are discarded. Target verification consumes the old anchor and the proposals. If L proposals survive, it commits those L tokens and one correction or bonus token. The new anchor is at $a + L + 1$. Keep target KVs only for positions below that new anchor. Features for the old anchor and the L retained proposals supply the next context append. The new anchor has not yet been processed by the target. End-of-sequence and length caps can truncate this last step.

Ideal greedy guarantee. Under exact target evaluation, deterministic tie-breaking, identical stopping rules, and correct causal cache maintenance, verification commits exactly the target’s greedy next token at every position. Induction on the committed prefix therefore gives the same greedy sequence as autoregressive decoding, for any maps W_i (Leviathan et al., 2023, Chen et al., 2023a). This is an idealized theoretical statement, not an empirical claim about numerical implementations.

3 TRAIN FOR COMPATIBILITY OR FOR SPECIALIZATION

3.1 CROSS-MODEL TRANSFER: RECONSTRUCT THE EXPECTED DRAFT INTERFACE

The new target first generates a response. Both frozen language models then process that saved text with causal context. With a shared tokenizer, features are paired at the same token positions. With different tokenizers, pairing uses eligible common text-prefix endpoints. Define $h_{p,i}^R$ as the reference feature and $c_p^R = N(\sum_i F_i h_{p,i}^R)$ as the reference draft context. The error compares each mapped vector with what the frozen drafter expects:

$$D(u, v) = \frac{\|u - v\|_2^2}{\|v\|_2^2 + 10^{-6}}, \quad \ell_{b,p}^{\text{DIR}} = \frac{1}{5} \sum_i D(W_i h_{b,p,i}^T, h_{b,p,i}^R) + D(c_{b,p}, c_{b,p}^R). \quad (5)$$

The example index is b . If S_b is its selected set of positions and B is the local number of examples, the loss is

$$\mathcal{L}_{\text{DIR}} = \frac{1}{B} \sum_{b=1}^B \frac{1}{|S_b|} \sum_{p \in S_b} \ell_{b,p}^{\text{DIR}}. \quad (6)$$

We call this **Drafter Interface Reconstruction (DIR)**. It is normalized squared reconstruction, not a new loss family. The first term constrains all five feature streams. The second constrains their fused, normalized effect. Equal example weighting prevents long responses from dominating. Its gradients stop at this interface, avoiding backward computation through the draft transformer and vocabulary head.

3.2 FINE-TUNED TARGETS: IMPROVE REACHABLE TOKEN PROPOSALS

For target specialization, the initial identity interface is usable. We generate labels with the active target and train through the frozen draft body. At response anchor a , masked slot $j \in \{1, \dots, 15\}$ predicts saved token $y_{ba,j} = x_{b,a+j}$. Let $q_{ba,j}(y)$ be its predicted probability and $m_{ba,j}$ indicate a valid response label. Define $r_{ba,j} = 1$ when the predicted argmax is $y_{ba,j}$ or the label is invalid, and zero otherwise. AUF uses

$$s_{ba,j} = \text{stopgrad} \prod_{k < j} r_{ba,k}, \quad \mathcal{L}_{\text{AUF}} = \frac{\sum_{b,a,j} m_{ba,j} s_{ba,j} [-\log q_{ba,j}(y_{ba,j})]}{\sum_{b,a,j} m_{ba,j} s_{ba,j}}. \quad (7)$$

The empty product is one. The first wrong proposal receives loss. Only later positions are excluded. The support is recomputed at each forward pass and treated as fixed during differentiation. AUF is from [Yang and Li \(2026\)](#). DFlash’s comparison objective instead uses $s_{ba,j} = \exp(-(j-1)/7)$ and retains every valid label ([Chen et al., 2026](#)). Chunked vocabulary losses sum numerators and denominators before local normalization. Distributed training averages four locally normalized microbatch gradients, not one global token ratio.

Why this distinction is testable. A frozen consumer can fail when its input coordinates change. DIR directly repairs that interface. For a fine-tuned target, returning exactly to base-model features need not improve specialized token proposals. AUF addresses which proposals contribute to a usable prefix. Neither argument proves objective superiority. Section 7 tests the explanation and reports contrary cases.

4 EXPERIMENTAL DESIGN

The main LoRA recipe uses 4,096 training examples, up to 32 anchors per example, one epoch, and 512 updates. Cross-model DIR uses 16,384 examples, fixed 25% prompt and response position coverage, three epochs, and 6,144 updates. Both use two L40S GPUs, two examples per GPU, and two accumulation rounds, giving batch eight. AdamW ([Loshchilov and Hutter, 2019](#)) uses learning rates 10^{-4} for token training and 10^{-3} for DIR, zero weight decay, gradient clipping at one, and 5% warmup followed by cosine decay. Trainable maps are FP32. The last checkpoint is used. Fitting time excludes response generation, feature extraction, and model setup. Complete recipes, precision settings, and source pins are in Appendix A.

5 RESULTS ACROSS SPECIALIZATIONS AND MODEL TRANSFERS

Standalone evaluation uses one L40S, BF16 Transformers/DFlash, greedy decoding, thinking disabled where supported, and natural stopping. Each workload has 128 prompts. The output cap is 2,048, except KiCad at 8,192. For request b , let o_b be output tokens and t_b measured generation time, including prefill. Report mean request throughput $v = n^{-1} \sum_b o_b / t_b$. Speedup is a ratio of these means. Native means the unadapted DFlash drafter for the evaluated target. These are throughput studies, not benchmark answer-accuracy measurements.

5.1 FINE-TUNED TARGETS: STRONG GAINS WITH CREDIBLE COMPETING CONTROLS

Table 1: Matched adaptation on Qwen3-4B specializations. Every cell uses 4,096 examples, 32 anchors, one epoch, and the same frozen target. Entries are percentage throughput gains over native DFlash. W is five full maps. Body is draft-body LoRA rank 128. ΔF trains an additive full fusion update.

Workload	W/AUF	W/Decay	Body/AUF	Body/Decay	ΔF /AUF	W fit min
GSM8K	37.58	36.84	36.41	34.82	32.36	2.66
KiCad	89.52	93.53	75.46	84.95	44.46	5.32
NanoCoder	8.07	3.11	9.82	8.65	8.15	2.76

The default five-map AUF models reach $4.51\times$, $8.06\times$, and $4.16\times$ autoregressive throughput. Body adaptation has 36.700M trainable parameters versus 32.768M for the maps. Decaying CE wins KiCad, and body AUF wins NanoCoder. The full 42-cell comparison includes joint maps and low-rank replacements (Appendix B). Their non-identity initialization differs from residual adaptation.

At 8B, adapting the *native 8B drafter* improves mean throughput by 35.1% on math and 6.2% on CodeAlpaca. Mapped rates are 173.88 and 162.13 tokens/s, or $4.59\times$ and $4.32\times$ AR. Fits take 4.06 and 4.02 minutes. These are two additional published target adapters, not the same adapters scaled to 8B.

5.2 CROSS-MODEL TRANSFER: MATHEMATICAL WORKLOADS TRANSFER BEST

Table 2: Four model transfers, one selected DIR recipe. The arrow identifies the original drafter’s model and the new target. Every cell gives mapped tokens/s followed by its AR-relative throughput ratio. Each workload has 128 prompts. All AR controls are measured in the corresponding standalone setup.

Original drafter → target	MATH	GSM8K	Code	Chat
T1 Qwen3 4B → 8B	220.26 / 5.82 $\times$	176.87 / 4.68 $\times$	127.61 / 3.40 $\times$	80.93 / 2.31 $\times$
T2 Qwen3 8B → 4B	216.37 / 5.49 $\times$	169.53 / 4.27 $\times$	130.83 / 3.30 $\times$	81.16 / 2.00 $\times$
T3 Llama 8B → 3B	145.43 / 2.35 $\times$	140.60 / 2.18 $\times$	118.57 / 1.88 $\times$	104.43 / 1.61 $\times$
T4 Qwen3 4B → Llama 8B	121.94 / 2.97 $\times$	90.84 / 2.21 $\times$	80.60 / 1.98 $\times$	60.52 / 1.48 $\times$

Training uses NuminaMath-derived prompts (Li et al., 2024a). Evaluation covers MATH test (Hendrycks et al., 2021), GSM8K test (Cobbe et al., 2021), LiveCodeBench (Jain et al., 2024), and UltraFeedback-derived instructions (Cui et al., 2023). These are four separate workloads. T1 recovers 100.2%, 99.4%, 84.2%, and 90.4% of native-drafter throughput. T2 recovers 96.4%, 95.6%, 76.2%, and 85.1%. T4 recovers 93.7%, 71.3%, 65.8%, and 52.9%. No native T3 drafter comparator is established in this study, so native-relative recovery is not assigned to T3.

T1, T2, and T3 fits take 24.91, 19.09, and 20.58 minutes on two GPUs. T4 records a 20-minute-33-second allocation including setup, not a comparable fitting-loop timer. T4 runtime includes its tokenizer bridge, whose exact boundary policies still require release.

Generalization and selection. The targets span Qwen3 and Llama (Yang et al., 2025, Grattafiori et al., 2024), with a separate fit per pair. Most fits use one seed. Training-disjoint evaluation cohorts were repeatedly inspected, so they are exploratory rather than untouched confirmation sets.

6 SHARED SERVING AND COMPARISON BOUNDARIES

A request’s target identity selects both its target LoRA and its folded draft fusion bank. One target backbone and one draft body serve the mixture. Selection follows individual request rows, not a global mutable matrix. The implementation uses vLLM (Kwon et al., 2023), alongside the target-adapter batching problem studied by Punica and S-LoRA (Chen et al., 2023b, Sheng et al., 2023).

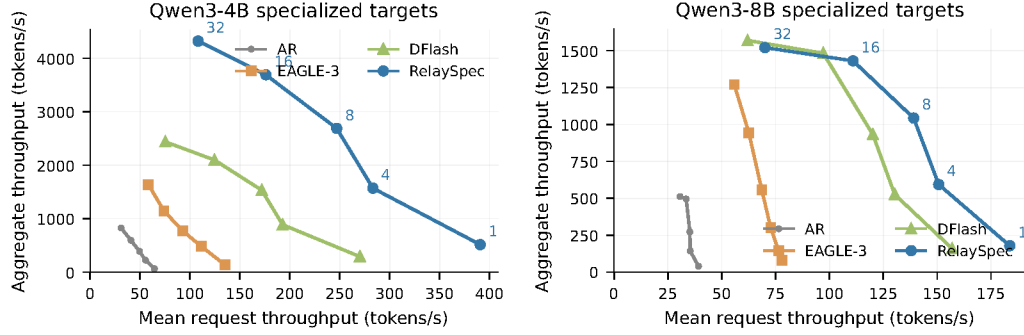


Figure 3: Serving throughput versus per-request speed. One L40S per engine, 384 requests at 4B and 256 at 8B. Labels on RelaySpec mark clients 1, 4, 8, 16, and 32. Both axes report tokens/s but use different time denominators. EAGLE-3 proposes three tokens in this serving configuration, versus fifteen for DFlash. The curves compare deployed configurations, not equal proposal budgets.

Table 3: Completed optimized serving measurements. Native DFlash and RelaySpec use the same 15-token proposal length. Gain is relative to native DFlash. Negative aggregate gains at 8B are retained.

Target	Quantity	1 client	4 clients	8 clients	16 clients	32 clients
4B	Mean-request gain (%)	44.60	46.93	43.75	41.12	43.77
4B	Aggregate gain (%)	77.03	76.82	75.24	76.05	77.23
8B	Mean-request gain (%)	17.02	15.66	15.76	14.22	13.03
8B	Aggregate gain (%)	12.20	12.85	12.03	-3.45	-3.09

At one client, the 4B mixture reaches 390.71 mean request tokens/s with RelaySpec, compared with 270.21 for native DFlash, 135.14 for EAGLE-3, and 64.39 for AR. The corresponding mean-throughput AR ratios are $6.07\times$, $4.20\times$, and $2.10\times$. At 32 clients, aggregate RelaySpec throughput reaches 4,323.96 tokens/s. The 8B negative aggregate gains show that improving average individual request rate need not improve mixture completion throughput.

Cross-model serving. A separate 52-cell study evaluates the selected DIR checkpoints. Every cell contains 128 requests. T1 uses clients 1, 8, 16, and 32. Its aggregate/native ratios range from 1.025 to 1.160 on MATH/GSM8K and 0.886 to 0.977 on Code/Chat. T2–T4 use 16 clients. T4 reaches only 0.364–0.630 of native aggregate throughput. T1 concurrent AR measurements are not available and are not inferred from standalone rates. Full absolute rates and available AR comparisons appear in Appendix D.

Controls on specialization. A separate earlier rank-56 residual fusion bank improves twelve-client throughput by 31.8–32.9% over native DFlash. A single pooled correction reaches 98.94–99.60% of separate-bank throughput with explicit target selection. Thus these workloads do not establish that every target needs a private draft expert. This is a different parameterization from the five full maps. A frozen TF-IDF/logistic classifier supplies identical target assignments across methods in the learned-routing control.

Fair comparator interpretation. Standalone EAGLE-3 uses its custom KV implementation with tree size 60, depth 7, and top- k 10 (Li et al., 2025). It has its own AR companion in a separate software environment. The serving comparison uses the same workload, not equal proposal budgets. No matched PARD-2 run is available. Published PARD-2 ratios on A100 and DFlash ratios on H200/B200 are literature context, not interchangeable denominators for our L40S measurements (An et al., 2026b, Chen et al., 2026).

7 WHY THE INTERFACE HELPS

7.1 CROSS-MODEL RECONSTRUCTION IMPROVES BEHAVIOR BEYOND ITS OWN LOSS

The T1 probe uses 8k-example fits. DIR uses 50% coverage and learning rate 10^{-3} . AUF and decay use 32 anchors and 10^{-4} . Training-feature context error is 0.136 for DIR, 0.940 for AUF, and 0.900 for decay. This is a recipe comparison, not a loss-only ablation.

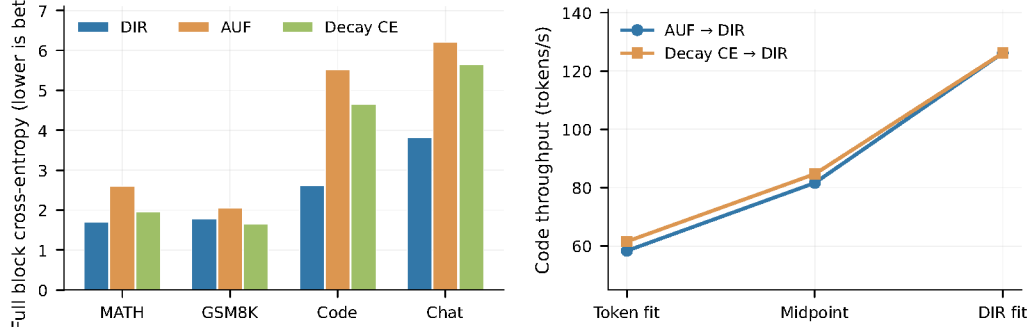


Figure 4: Controlled inputs and live interventions. Left: full block cross-entropy on the same 64 blocks from eight held-out examples per task. Right: live Code throughput on 128 prompts. A midpoint averages all five token-trained and DIR matrices without retraining. Neither chart is a benchmark answer-accuracy measure.

On held-out blocks, Code CE falls from 5.508 with AUF to 2.604 with DIR. Replacing W_i^{token} by $(W_i^{\text{token}} + W_i^{\text{DIR}})/2$, without retraining, improves live Code throughput by 39.9% for AUF and 37.6% for decay. Accepted length rises from 2.346 to 3.251 and from 2.449 to 3.337. This supports the role of the interface through the same frozen body, but changes all layers together.

Counterexamples refine the explanation. All four warm-AUF refinements improve GSM8K but reduce other workloads. The unseen-8k fit reaches 1.0606 times native GSM8K throughput, while Code falls from 0.8322 to 0.7051. A rectangular-identity retry with higher learning rate improves Code/Chat by 19–25%, without closing the DIR gap. It cannot isolate initialization from learning rate.

7.2 SPECIALIZATION PRESERVES THE ORIGINAL PATH AND CHANGES DIRECTION

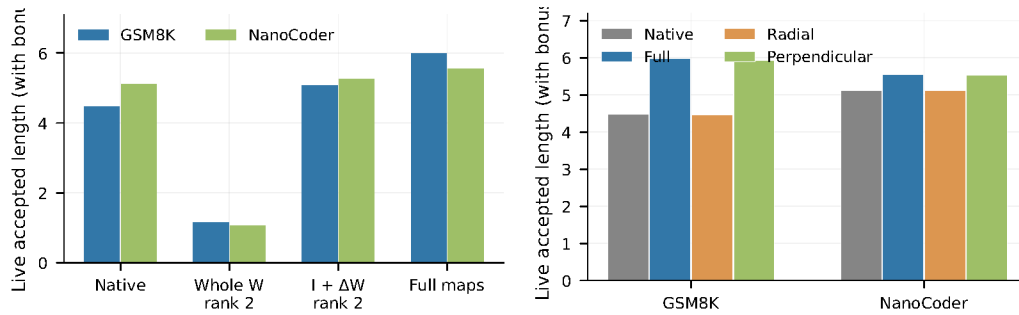


Figure 5: Live interventions on 4B specialization maps. Left: whole-map truncation removes the identity path. Truncating its change preserves that path. Right: matched controls isolate radial and perpendicular components. Each live test has 128 prompts.

Write $W_i = I + \Delta_i$, $z_0 = \sum_i F_i h_i$, and $\delta = \sum_i F_i \Delta_i h_i$. The radial component is $\delta_{\parallel} = (\delta^{\top} z_0) z_0 / \max(\|z_0\|^2, 10^{-20})$, and $\delta_{\perp} = \delta - \delta_{\parallel}$. RMSNorm largely removes positive rescaling. Radial-only accepted length stays near native, while perpendicular-only approaches the full correction (Figure 5). Literal reversal of the target LoRA is not supported: mean cosine between mapper correction and target-induced shift is -0.0429 on math and $+0.1248$ on NanoCoder, close to shuffled controls. The maps learn useful conditioning, not a demonstrated inverse of target fine-tuning.

8 SUPERVISION SCALING AND COMPUTATIONAL ACCOUNTING

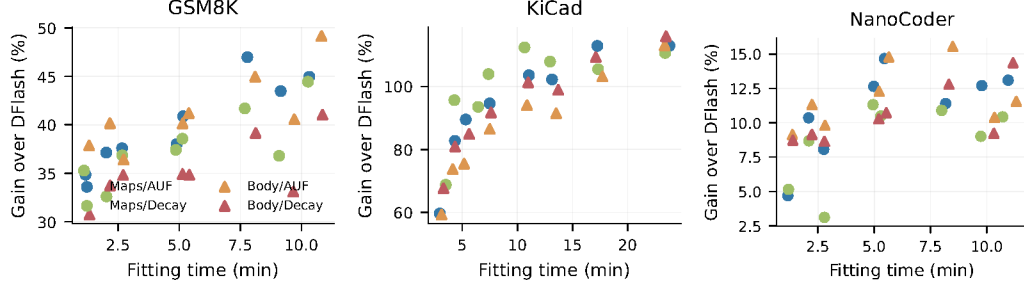


Figure 6: All 96 matched supervision-budget settings. Each point is an independently fitted one-epoch checkpoint. We sweep anchors 8/24/32/64/128 at 4,096 examples and example presentations 4,096/8,192/12,288/16,384 at 32 anchors. The intersecting cell is counted once. Fitting excludes feature preparation. All methods and dominated points remain visible.

More supervision helps unevenly. At 4,096 examples, increasing five-map AUF anchors from 8 to 128 raises native-relative gain from 34.85% to 43.47% on GSM8K and from 59.65% to 102.24% on KiCad. At 32 anchors, the highest observed gains are 46.98%, 112.96%, and 14.67% across math, KiCad, and NanoCoder, with fitting times 7.78, 23.78, and 5.46 minutes. These are post hoc maxima. Serving uses the fixed 4,096-by-32 checkpoints, not those maxima. Body adaptation wins several larger-budget cells.

Presentation count is not always unique data. GSM8K exhausts 4,096 compatible unique examples before repetition. KiCad has 15,366 and NanoCoder 18,016 unique cached examples. Requested anchors are capped by valid response positions. Appendix C retains the full grid and reported prompt-bootstrap intervals.

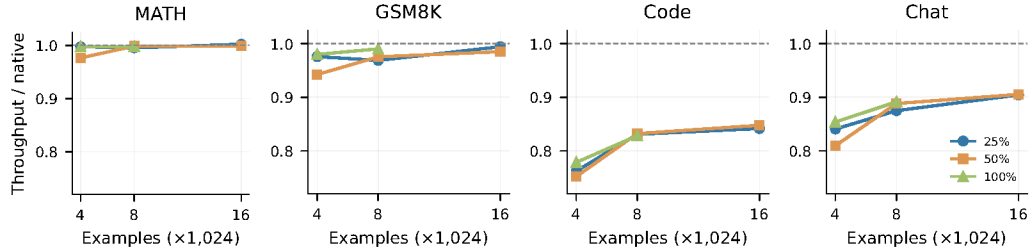


Figure 7: Cross-model data and position coverage. T1 DIR, three epochs. Lines connect measured observations, not fitted scaling laws. The 16k/100% setting was not measured and is not plotted. Ratios use native mean request throughput.

At 16,384 examples, 25% position coverage takes 24.91 minutes versus 37.21 at 50%, a 33.1% reduction. Accepted length differs by less than 0.3% on every workload. More examples help Code/Chat more consistently than denser position coverage. These eight settings support a practical budget choice, not a power law or an asymptotic scaling prediction.

Parameterization changes learning. If H_i is the gradient of effective fusion $G_i = F_i W_i$, gradient descent on W_i induces $\Delta G_i = -\eta F_i F_i^\top H_i$, not the direct step $-\eta H_i$. Here η is step size. Actual Adam updates also differ (Appendix E). Different updates do not prove final-checkpoint superiority.

Costs that folding does and does not remove. A folded projection costs about $10d_t d_d$ FLOPs per new context position, counting multiply-add as two operations. A same-width BF16 bank stores 65.536 MB at 4B or 167.772 MB at 8B. Draft KV requires 20,480 bytes per retained context position per request. These analytic counts exclude target KV, temporary activations, and allocator overhead. Approximate decode time per emitted token is $(C_D + C_V)/\tau$, with draft time C_D , verification time C_V , and committed length τ . Prefill and scheduling add latency. These are analytic costs, not profiler measurements or peak memory.

9 POSITIONING, LIMITS, AND IMPACT

Table 4: What changes, and when the methods are useful. This is an architectural comparison, not an unmeasured numerical ranking. RelaySpec requires an offline fit for each new model pair.

Method	What is learned or changed	Use of target features	Distinction from RelaySpec
DFlash	Parallel masked-block drafter	Context in every draft layer	Establishes the frozen drafting architecture we adapt
PARD	Standalone small model adapted to parallel prediction	Target-independent mode	Can transfer without our per-pair feature fit
PARD-2	Draft training with confidence weighting and stochastic feature injection	Optional target conditioning	Trades conditioning for zero-shot within-family flexibility
EDA	Shared and private draft components, selected adaptation data	Adapts to fine-tuned targets	Closest adaptation problem, not reproduced end to end here
RepSpec	Redundant linear training structures, merged at export	Depends on its draft design	Weight folding itself is not our novelty
RelaySpec	Five input maps, frozen pretrained draft body	New target features remain active	Tests interface-only transfer and specialization in one framework

PARD-2 can disable target conditioning for within-family transfer without another fit (An et al., 2026b). We instead retain that conditioning and fit only its interface. EDA, direct draft alignment, and OmniDraft already study retargeting (Lin et al., 2026, Goel et al., 2024, Zhou et al., 2023, Ramakrishnan et al., 2025). Our contribution is the measured ability to localize adaptation in a frozen-body interface, not the adaptation problem itself.

Proposal architectures, tree verification, and asynchronous execution address other parts of decoding (Li et al., 2025, Cai et al., 2024, Miao et al., 2023, Kumar et al., 2026). They are not experimentally combined with our maps. Model stitching and representation finetuning are broader precedents for adapting internal interfaces (Bansal et al., 2021, Wu et al., 2024). Similarity alone does not prove downstream utility (Kornblith et al., 2019), motivating our live interventions. Appendix F expands these comparisons.

Scope. The architecture is tested on DFlash. Cross-family transfer is weaker than within-Qwen transfer. Larger targets, non-greedy settings, multi-seed confirmation, and complete adaptation cost remain unmeasured. No matched PARD-2 or full EDA reproduction establishes superiority. Complete manifests and cross-tokenizer runtime code remain release requirements.

Reserved: A100 and DGX Spark. Add matched checkpoints, prompts, precision, proposal budgets, and client counts. Report absolute throughput and latency. Cross-hardware results are pending.

Reserved: measured FLOPs and overhead. Add draft, projection, verification, and tokenizer/dispatch timings. Report operations per emitted token and peak memory separately from analytic counts.

Reserved: ultra-long context. Add fixed-checkpoint context sweeps and separate prefill/decode timings on LongBench and RULER (Bai et al., 2023, Hsieh et al., 2024). LongAlign-based retraining must be a separate condition (Bai et al., 2024).

10 CONCLUSION

A frozen DFlash body can serve changed targets through a fitted input interface. Reconstruction is a strong cross-model recipe, while AUF is useful for specialization. The contribution is this adaptation strategy, supported by measured gains and functional interventions, with its workload and deployment limits explicit.

IMPACT STATEMENT

The intended benefit is to make existing speculative drafters useful across more deployments without retraining their transformer weights. This can lower the engineering and fitting burden of specialized inference and make acceleration accessible to teams with modest adaptation budgets. Reduced fitting parameters do not by themselves prove lower total energy use. Generating and storing training features has a cost, and faster inference can also increase total demand. The paper reports throughput, not environmental savings.

The target model remains responsible for final token commitment. Faster generation does not improve the truthfulness, safety, or engineering validity of its answers. In particular, generated code and KiCad outputs must still pass the validation required by their application. Feature caches can contain sensitive information about prompts, so deployments using private data should apply appropriate retention and access controls. The experiments described here use existing model releases and public task data and do not involve human-subject studies.

REPRODUCIBILITY STATEMENT

The main text defines the maps, frozen parameters, exact attention visibility, per-layer projection sharing, cache updates, objective support, and metric denominators. The appendix records optimizer settings, available model and dataset pins, full matched architecture and budget tables, and current serving measurements. The manuscript package contains curated result CSVs, figure-generation scripts, and a minimal mathematical reference implementation. That implementation is a new illustration of the equations, not the original benchmark engine. Original customized runtime code, complete prompt manifests, and exact cross-tokenizer boundary policies are not included in the supplied experimental records. Those release items remain necessary for independent numerical replication. No new GPU experiment was performed in preparing this manuscript.

AI USE STATEMENT

AI assistance was used to prepare and revise this manuscript, inspect supplied experimental reports, search related literature, generate plotting and reference code, and check numerical consistency. The supplied experiment reports also describe AI-assisted experimental design, implementation, debugging, and analysis. The manuscript preparation process did not independently reproduce the GPU experiments. The authors remain responsible for verifying all experimental claims, code, source attributions, and final submission material.

REFERENCES

- Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebr'on, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023. URL <https://arxiv.org/abs/2305.13245>.
- Zihao An, Huajun Bai, Ziqiong Liu, Dong Li, and Emad Barsoum. PARD: Accelerating LLM inference with low-cost parallel draft model adaptation. *arXiv preprint arXiv:2504.18583*, 2026a. URL <https://arxiv.org/abs/2504.18583>.
- Zihao An, Taichi Liu, Ziqiong Liu, Dong Li, Ruofeng Liu, and Emad Barsoum. PARD-2: Target-aligned parallel draft model for dual-mode speculative decoding. *arXiv preprint arXiv:2605.08632*, 2026b. URL <https://arxiv.org/abs/2605.08632>.
- Yushi Bai, Xin Lv, Jiajie Zhang, Yuze He, Ji Qi, Lei Hou, Jie Tang, Yuxiao Dong, and Juanzi Li. LongAlign: A recipe for long context alignment of large language models. *arXiv preprint arXiv:2401.18058*, 2024. URL <https://arxiv.org/abs/2401.18058>.
- Yushi Bai et al. LongBench: A bilingual, multitask benchmark for long context understanding. *arXiv preprint arXiv:2308.14508*, 2023. URL <https://arxiv.org/abs/2308.14508>.

- Yamini Bansal, Preetum Nakkiran, and Boaz Barak. Revisiting model stitching to compare neural representations. *arXiv preprint arXiv:2106.07682*, 2021. URL <https://arxiv.org/abs/2106.07682>.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024. URL <https://arxiv.org/abs/2401.10774>.
- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023a. URL <https://arxiv.org/abs/2302.01318>.
- Jian Chen, Yesheng Liang, and Zhijian Liu. DFlash: Block diffusion for flash speculative decoding. *arXiv preprint arXiv:2602.06036*, 2026. URL <https://arxiv.org/abs/2602.06036>.
- Lequn Chen, Zihao Ye, Yongji Wu, Danyang Zhuo, Luis Ceze, and Arvind Krishnamurthy. Punica: Multi-tenant LoRA serving. *arXiv preprint arXiv:2310.18547*, 2023b. URL <https://arxiv.org/abs/2310.18547>.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Łukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021. URL <https://arxiv.org/abs/2110.14168>.
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, Yuan Ni, Guotong Xie, Zhiyuan Liu, and Maosong Sun. UltraFeedback: Boosting language models with scaled AI feedback. *arXiv preprint arXiv:2310.01377*, 2023. URL <https://arxiv.org/abs/2310.01377>.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher R’e. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, Ahmed A. Aly, Beidi Chen, and Carole-Jean Wu. LayerSkip: Enabling early exit inference and self-speculative decoding. *arXiv preprint arXiv:2404.16710*, 2024. URL <https://arxiv.org/abs/2404.16710>.
- Raghav Goel, Mukul Gagrani, Wonseok Jeon, Junyoung Park, Mingu Lee, and Christopher Lott. Direct alignment of draft model for speculative decoding with chat-fine-tuned LLMs. *arXiv preprint arXiv:2403.00858*, 2024. URL <https://arxiv.org/abs/2403.00858>.
- Aaron Grattafiori et al. The Llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL <https://arxiv.org/abs/2407.21783>.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*, 2021. URL <https://arxiv.org/abs/2103.03874>.
- Taekyung Heo, Rasoul Shafipour, Ritchie Zhao, Maximilian Golub, Mohammad Mahdi Kamani, Ritika Borkar, Makesh Tarun Chandran, Pantea Zardoshti, and Bitu Darvish Rouhani. Cross-model KV cache transfer in LLM families: A closed-form linear mapping for prefill reuse. *arXiv preprint arXiv:2608.03893*, 2026. URL <https://arxiv.org/abs/2608.03893>.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekish, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What’s the real context size of your long-context language models? *arXiv preprint arXiv:2404.06654*, 2024. URL <https://arxiv.org/abs/2404.06654>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2022. URL <https://arxiv.org/abs/2106.09685>.

- Feiye Huo, Jianchao Tan, Jiahao Liu, Zixu Jiang, Jiacheng Li, Jingang Wang, Xunliang Cai, and Shengli Sun. RepSpec: Structural re-parameterized draft model training for speculative decoding. In *International Conference on Learning Representations*, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/hash/d70ea003729b440b89a2f958a5554clf-Abstract-Conference.html.
- Naman Jain et al. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. *arXiv preprint arXiv:1905.00414*, 2019. URL <https://arxiv.org/abs/1905.00414>.
- Tanishq Kumar, Tri Dao, and Avner May. Speculative speculative decoding. *arXiv preprint arXiv:2603.03251*, 2026. URL <https://arxiv.org/abs/2603.03251>.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. *arXiv preprint arXiv:2309.06180*, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. *arXiv preprint arXiv:2211.17192*, 2023. URL <https://arxiv.org/abs/2211.17192>.
- Jia Li, Edward Beeching, Lewis Tunstall, Ben Lipkin, Roman Soletskyi, Shengyi Costa Huang, Kashif Rasul, Longhui Yu, Albert Jiang, Ziju Shen, Zihan Qin, Bin Dong, Li Zhou, Yann Fleureau, Guillaume Lample, and Stanislas Polu. NuminaMath. Public dataset, 2024a. URL <https://huggingface.co/datasets/AI-MO/NuminaMath-CoT>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024b. URL <https://arxiv.org/abs/2401.15077>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. *arXiv preprint arXiv:2406.16858*, 2024c. URL <https://arxiv.org/abs/2406.16858>.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025. URL <https://arxiv.org/abs/2503.01840>.
- Luxi Lin, Zhihang Lin, Zhanpeng Zeng, Yuhao Chen, Qingyu Zhang, Jixiang Luo, Xuelong Li, and Rongrong Ji. Efficiently aligning draft models via parameter- and data-efficient adaptation. *arXiv preprint arXiv:2603.09527*, 2026. URL <https://arxiv.org/abs/2603.09527>.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2019. URL <https://arxiv.org/abs/1711.05101>.
- Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, Chunan Shi, Zhuoming Chen, Daiyaan Arfeen, Reyna Abhyankar, and Zhihao Jia. SpecInfer: Accelerating generative large language model serving with tree-based speculative inference and verification. *arXiv preprint arXiv:2305.09781*, 2023. URL <https://arxiv.org/abs/2305.09781>.
- Giovanni Monea, Armand Joulin, and Edouard Grave. PaSS: Parallel speculative sampling. *arXiv preprint arXiv:2311.13581*, 2023. URL <https://arxiv.org/abs/2311.13581>.
- Ramchalam Kinattinkara Ramakrishnan, Zhaocong Yuan, Shaojie Zhuo, Chen Feng, Yicheng Lin, Chenzheng Su, and Xiaopeng Zhang. OmniDraft: A cross-vocabulary, online adaptive drafter for on-device speculative decoding. *arXiv preprint arXiv:2507.02659*, 2025. URL <https://arxiv.org/abs/2507.02659>.

- Noam Shazeer. GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020. URL <https://arxiv.org/abs/2002.05202>.
- Ying Sheng, Shiyi Cao, Dacheng Li, Coleman Hooper, Nicholas Lee, Shuo Yang, Christopher Chou, Banghua Zhu, Lianmin Zheng, Kurt Keutzer, Joseph E. Gonzalez, and Ion Stoica. S-LoRA: Serving thousands of concurrent LoRA adapters. *arXiv preprint arXiv:2311.03285*, 2023. URL <https://arxiv.org/abs/2311.03285>.
- Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021. URL <https://arxiv.org/abs/2104.09864>.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *arXiv preprint arXiv:1706.03762*, 2017. URL <https://arxiv.org/abs/1706.03762>.
- Zhengxuan Wu, Aryaman Arora, Zheng Wang, Atticus Geiger, Dan Jurafsky, Christopher D. Manning, and Christopher Potts. ReFT: Representation finetuning for language models. *arXiv preprint arXiv:2404.03592*, 2024. URL <https://arxiv.org/abs/2404.03592>.
- Zilin Xiao, Hongming Zhang, Tao Ge, Siru Ouyang, Vicente Ordonez, and Dong Yu. ParallelSpec: Parallel drafter for efficient speculative decoding. *arXiv preprint arXiv:2410.05589*, 2024. URL <https://arxiv.org/abs/2410.05589>.
- An Yang et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Tianjian Yang and Meng Li. Spec-AUF: Accept-until-fail training under train-inference misalignment for masked block drafters. *arXiv preprint arXiv:2607.01893*, 2026. URL <https://arxiv.org/abs/2607.01893>.
- Biao Zhang and Rico Sennrich. Root mean square layer normalization. *arXiv preprint arXiv:1910.07467*, 2019. URL <https://arxiv.org/abs/1910.07467>.
- Jiebin Zhang, Zhenghan Yu, Song Liu, Eugene J. Yu, Zheng Li, Dawei Zhu, Jiangshan Duo, Weimin Xiong, Yifan Song, Guanghua Yu, Jianchen Zhu, and Sujian Li. DFlare: Scaling up draft capacity for block diffusion speculative decoding. *arXiv preprint arXiv:2606.02091*, 2026. URL <https://arxiv.org/abs/2606.02091>.
- Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. *arXiv preprint arXiv:2312.07104*, 2024. URL <https://arxiv.org/abs/2312.07104>.
- Yongchao Zhou, Kaifeng Lyu, Ankit Singh Rawat, Aditya Krishna Menon, Afshin Rostamizadeh, Sanjiv Kumar, Jean-François Kagy, and Rishabh Agarwal. DistillSpec: Improving speculative decoding via knowledge distillation. *arXiv preprint arXiv:2310.08461*, 2023. URL <https://arxiv.org/abs/2310.08461>.

A DETAILED REPRODUCTION CONTRACT

A.1 WHAT THE PRESENT PACKAGE REPRODUCES

This package reconstructs tables and figures from the attached experimental summaries. It also supplies an independently written mathematical implementation of the map, folding operation, two losses, and training attention mask. It does not contain the original custom training jobs, target-generated feature tensors, exact evaluation prompts, or the original serving engine. Source hashes in `provenance.json` identify the attached records used. The distinction matters: reproducing a table from a report is not reproducing the underlying GPU measurement.

A.2 MODEL AND FEATURE INVENTORY

Table 5: Exact principal architecture dimensions. Each map converts a new target stream into the width of the retained draft body. Model sizes label the language models, not the much smaller draft transformers.

Setting	d_t	d_d	Maps	Trainable entries	Initialization
4B specialization	2560	2560	5	32,768,000	Identity
8B specialization	4096	4096	5	83,886,080	Identity
T1 Qwen3 4B $\rightarrow$ 8B	4096	2560	5	52,428,800	Xavier
T2 Qwen3 8B $\rightarrow$ 4B	2560	4096	5	52,428,800	Xavier
T3 Llama 8B $\rightarrow$ 3B	3072	4096	5	62,914,560	Xavier
T4 Qwen3 $\rightarrow$ Llama	4096	2560	5	52,428,800	Xavier

Qwen3-4B and Qwen3-8B provide streams from zero-based transformer layers 1, 9, 17, 25, and 33. Llama-3.1-8B uses 1, 8, 15, 22, and 29. Llama-3.2-3B uses 1, 7, 13, 19, and 25. Post-layer hidden-state array indices are one larger when that array includes the embedding output. Pairing is by stream slot, not by equal numerical layer index. The Qwen mask ID is 151669. The Llama reference draft uses mask ID 128002. The Qwen vocabulary has 151936 entries.

The 4B draft has width 2560, five transformer layers, and MLP width 9728. Its embedding and vocabulary head are tied frozen reference weights. The corresponding 8B draft has width 4096, five layers, and MLP width 12288, with untied frozen embedding/head weights. Both have 32 query heads, eight KV heads, and head dimension 128. For cross-model transfer, the embedding and vocabulary head remain those of the original drafter. They are not silently replaced by the new target’s weights. Both language models remain frozen.

Table 6: Pinned principal model releases. Long revisions are broken typographically but should be copied from the machine-readable provenance file.

Repository	Revision or weight hash
Qwen/Qwen3-4B	1cfa9a7208912126459214e8b04321603b3df60c
Qwen/Qwen3-8B	b968826d9c46dd6066d109eabc6255188de91218
z-lab/Qwen3-4B-DFlash-b16	b74e3a329c4d963783143b1e970d95b002be72bd
z-lab/Qwen3-8B-DFlash-b16	9b41424b7109f9c5413454f481b09a82b85333f4
unsloth/Llama-3.1-8B-Instruct	4699cc75b550f9c6f3173fb80f4703b62d946aa5
unsloth/Llama-3.2-3B-Instruct	006f5dcd1393c3add266de40994ba96225e9689d

Target adapters. At 4B the specializations are `witcheer/qwen3-4b-gsm8k-grpo`, `AbijahKaj/qwen3-4b-kicad-netlist`, and `usernamebetter/nanocoder-v1`. Math is pinned at `f52e1ed1b3accd404d37a9621db0536dd99bdfc5` and `NanoCoder` at

f078bfc2722bdb7bf644906cc7d81662ab3de9d2. An exact KiCad adapter revision is not stated in the supplied summary. Its acquisition manifest remains a release requirement.

At 8B, math uses sumitdotml/lora-and-friends, revision 6c91e19043bb9c784b7f982f488a134f043dd944, subfolder checkpoints/best-checkpoints/attention_only/seed-0/step-3169. It is a rank-eight, alpha-32 q/k/v/o target adapter published as OpenMathSFT adaptation. The coding target uses lovepon/Qwen3-8B-code_alpaca-lora, revision c0b5f6ed334666be14a09faf1d1363029ae13a76, rank eight and alpha sixteen on q/v. These target adapters were not trained by the draft-adaptation experiment.

A.3 DATA CONSTRUCTION AND SAMPLE COUNTS

For 4B math, openai/gsm8k is pinned at 740312add88f781978c0658806c59bc2815b9866. Python Random(42), stripped-question hashes, and disjoint training/test splits select 4,096 training and 128 evaluation questions. The user prompt asks for brief reasoning and an answer ending in ##### <answer>. It is rendered through the base chat template with thinking disabled. The target generates each training continuation once, capped at 4,096 response tokens. Human reference answers are not the token training labels, and generated responses are not correctness-filtered.

KiCad uses AbijahKaj/kicad-netlist-sft-dataset, revision 52ce8608c3c0fd2f1b6b68e93b8784b88bc92373. Publisher-style ordering, chat encoding, and an 8,192-token filter retain 16,738 rows. A 2% split with seed 42 yields 16,403 training and 335 validation rows. The default fit uses 4,096 deduplicated direct, non-tool training prompts. The 128 direct validation prompts exclude exact training-prompt matches. This is a reconstruction of a publisher-style split, not an independently certified circuit benchmark.

NanoCoder uses HuggingFaceH4/CodeAlpaca_20K, revision 798c567f69c8f4b12fc191015e59ee34e9afe00d. The stored split is shuffled with seed 20260909. Stripped-prompt duplicates and test prompts present in training are excluded. A 3,072-token prompt limit applies. The default cohort selects the shortest and longest eligible prompts before filling from the shuffled pool. A system message identifies NanoCoder as a full-stack engineering and debugging assistant. The complete cached pool contains 18,016 unique training examples.

Cross-model training is described as NuminaMath-derived (Li et al., 2024a). The original manifest is needed to reproduce its exact public-data subset. T1’s 4k and 8k subsets are nested inside 16k. T2/T3 combine 4,096 compatible existing records and 12,288 new records. The earlier T3 subset’s generation cap is not specified in the supplied summary. Pairing uses full causal feature computation before positions are subsampled. Quarter-coverage positions are fixed across epochs and nested within the half-coverage sample. Sampling is applied separately to prompt and response segments.

Every main cross-model evaluation task has 128 prompts. MATH refers to the MATH test workload, not an assumed 500-example MATH-500 cohort. Code refers to the selected LiveCodeBench workload, not HumanEval. Chat uses UltraFeedback-derived instructions, not MT-Bench. No evaluation request count in this paper is inferred from a benchmark’s usual size. The one-client specialized 4B serving experiment contains 384 total requests, one 128-prompt cohort per domain.

A.4 OPTIMIZATION AND LOSS REDUCTION

All primary five-map and body comparisons use AdamW with betas (0.9, 0.999), optimizer epsilon 10^{-8} , zero weight decay, norm clipping at one, and seed 42. Same-model token training uses BF16 forward autocast and FP32 trainables. Cross-model reconstruction computes the feature loss in FP32 with TF32 disabled in the reported fitting path. Two GPUs, two examples per GPU, and two accumulated microbatches give eight examples per optimizer update. The learning rate warms up for 5% of updates, then follows cosine decay. The final checkpoint is selected without a development-set search.

Eligible anchors have both x_a and x_{a+1} inside the response. Sample at most 32 distinct eligible anchors uniformly without replacement and sort them for block construction. Blocks may overlap. Positions beyond response end are masked out. The anchor itself contributes no label. Predict x_{a+j} at offset j , without a further label shift. Prompt features are visible only as causal context.

Let $L_{g,b}$ be a locally normalized two-example loss on GPU g and accumulation round b . The actual effective update averages

$$L_{\text{step}} = \frac{1}{4} \sum_{g=1}^2 \sum_{b=1}^2 L_{g,b}. \quad (8)$$

This differs from pooling all valid labels across eight examples when label counts differ. Vocabulary cross-entropy is processed in two-block chunks. Chunk numerators and denominators are summed before local division. AUF’s detached support and decay’s exponential weights are included in their own denominators. The contrast between DIR and token objectives also changes supervision coverage and backward path. It is not a loss-only randomized trial.

A.5 INFERENCE ALGORITHM AND CODE-LEVEL CONVENTIONS

The inherited attention is based on the Transformer computation (Vaswani et al., 2017) with grouped KV heads, rotary positions, and RMS normalization. The exact greedy same-tokenizer state transition is:

Initialize. Prefill target on the prompt. Its next token becomes the clean anchor at position a . Target KV contains positions $p < a$. Keep five target feature streams for that committed prefix. Start each draft context cache empty.

Draft. Fold and normalize newly available context. In every draft layer, append its projected context keys/values, using true absolute positions. Form one anchor-plus-15-mask block. Query the persistent context and this block using the visibility mask in Figure 2. Predict 15 candidates with one forward pass. Crop the draft cache to remove the entire temporary block.

Verify. Run the target on the old anchor followed by the 15 candidate tokens. Find the retained candidate prefix of length L . Commit that prefix and one target correction or bonus token, subject to stopping and length caps.

Advance. Set $a' = a + L + 1$. Crop the target cache to positions below a' . Save target features for the old anchor and the L retained candidates. These $L + 1$ feature positions are the next context append. The new anchor token is known but has not yet been processed by the target. Repeat.

Persistent draft entries always come from target-context projections. The unverified block states are not a substitute for target features on the next iteration. In heterogeneous serving, request IDs must accompany context rows, feature-bank selection, cache ownership, and sequence reordering. Each target adapter and folded bank is selected by the same request identity.

T4 needs text-prefix alignment and a tokenizer bridge. The supplied report states that the bridge is timed, but does not contain its complete implementation. Re-tokenization boundaries, invalid partial-byte handling, special-token conversion, cache invalidation, and fallback rules must be released rather than invented. The same-tokenizer pseudocode above must not be mistaken for a complete T4 reproduction.

A.6 TIMING BOUNDARIES AND SOFTWARE

Standalone DFlash uses package 0.1.0 with reported PyTorch 2.13.0+cu129, Transformers 5.16.1, and PEFT 0.20.0. Native vLLM reports version 0.28.0+cu129. These are the recorded experimental environments, not a recommendation to substitute an arbitrary later release. SpecForge revision 953d43a0c1c0f5e32989dc43f91ce5fc2d9ddfef defines the inherited block-training contract. The custom trainable-parameter selection and AUF code are additional modifications.

For standalone timing, two 64-token training-prompt warmups precede measurement. CUDA synchronization brackets generation. Timing excludes loading, tokenization, warmup, and serialization. It includes target prefill and decoding. Targets with LoRA are statically merged for this path. In serving, compilation mode three, automatic scheduling, FlashAttention, capacity 32, a 32,768-token batching budget, and memory fraction 0.8 define the optimized cross-model setup. Prefix caching is reset before each cell. T1–T3 use the V1 runner and T4 uses its V2 bridge. No measured JIT event occurs in the retained 52 cross-model cells. FlashAttention reduces attention memory traffic (Dao et al., 2022), but does not remove the draft-interface bookkeeping.

EAGLE-3’s standalone implementation uses its separate custom-KV environment with Transformers 4.53.3 and PEFT 0.15.2, tree size 60, depth seven, and top- k ten. The Qwen compatibility patch respects the configured head dimension 128 rather than deriving it by dividing hidden width by head count. Each such environment has its own AR companion. Serving EAGLE uses three speculative tokens. The main comparisons therefore do not infer an equal-budget ranking over all EAGLE configurations.

B COMPLETE MATCHED ARCHITECTURE COMPARISONS

The seven parameterizations use the same 4B target, data, example order, anchors, training duration, and optimizer recipe within each objective. Five full W_i maps are initialized to identity. A joint full map mixes the concatenated five streams before the frozen fusion, giving $25d_d^2 = 163.84\text{M}$ trainable entries. A direct full fusion residual learns $F + \Delta F$ from a zero residual with 32.768M entries.

The low-rank replacement controls use $W_i = B_i A_i$, with $B_i \in \mathbb{R}^{d_d \times r}$ and $A_i \in \mathbb{R}^{r \times d_d}$. At 4B their initialization is a rank- r PCA projector constructed from training features. It is not $I + B_i A_i$. Their count is $10d_d r$, giving 0.7168M, 1.4336M, and 3.2768M at ranks 28, 56, and 128. These controls ask whether a restricted replacement works, not whether a small residual correction suffices.

The body LoRA baseline uses rank 128, alpha 256, zero dropout, and q/k/v/o/gate/up/down projections in all five draft layers. Fusion, norms, embedding, and vocabulary head remain frozen. It has 36.700M trainable entries. Its static updates are merged before standalone evaluation. The full maps and body baseline both differentiate through the frozen computations needed by the token loss. Trainable parameter savings alone are not proportional training-compute savings.

Table 7: All 14 matched architecture/objective cells on GSM8K. Gain is percent over native DFlash. AR is the mean-throughput ratio to the matched autoregressive control. Fitting time uses two GPUs and excludes cache preparation.

Architecture	Loss	Tok/s	Gain	AR	Accepted	Params M	Fit min
Five full maps	AUF	174.90	37.58	4.506	5.992	32.768	2.66
Full ΔF	AUF	168.27	32.36	4.335	5.809	32.768	2.60
Body LoRA r128	AUF	173.42	36.41	4.468	5.937	36.700	2.72
Replacement r28	AUF	72.39	-43.06	1.865	2.382	0.717	2.52
Replacement r56	AUF	99.26	-21.93	2.557	3.213	1.434	2.54
Replacement r128	AUF	131.24	3.24	3.382	4.339	3.277	2.47
Body LoRA r128	Decay	171.40	34.82	4.416	5.754	36.700	2.71
Full ΔF	Decay	168.44	32.49	4.340	5.621	32.768	2.67
Five full maps	Decay	173.97	36.84	4.482	5.780	32.768	2.67
Replacement r128	Decay	127.70	0.45	3.290	4.223	3.277	2.45
Replacement r28	Decay	70.44	-44.59	1.815	2.278	0.717	2.44
Replacement r56	Decay	97.19	-23.55	2.504	3.104	1.434	2.45
Joint full map	AUF	158.06	24.33	4.072	5.064	163.840	3.41
Joint full map	Decay	152.08	19.63	3.919	4.987	163.840	3.39

Table 8: All 14 matched architecture/objective cells on KiCad. Gain is percent over native DFlash. AR is the mean-throughput ratio to the matched autoregressive control. Fitting time uses two GPUs and excludes cache preparation.

Architecture	Loss	Tok/s	Gain	AR	Accepted	Params M	Fit min
Five full maps	AUF	309.56	89.52	8.057	11.259	32.768	5.32
Full ΔF	AUF	235.96	44.46	6.142	8.777	32.768	5.08
Body LoRA r128	AUF	286.59	75.46	7.459	10.495	36.700	5.17
Replacement r28	AUF	184.28	12.82	4.796	6.618	0.717	5.18
Replacement r56	AUF	220.72	35.13	5.745	8.119	1.434	4.99
Replacement r128	AUF	258.69	58.37	6.733	9.456	3.277	4.82
Body LoRA r128	Decay	302.10	84.95	7.863	10.875	36.700	5.63
Full ΔF	Decay	244.52	49.70	6.364	8.901	32.768	6.00
Five full maps	Decay	316.11	93.53	8.227	11.375	32.768	6.44
Replacement r128	Decay	266.09	62.91	6.926	9.354	3.277	4.64
Replacement r28	Decay	172.99	5.91	4.503	6.412	0.717	5.65
Replacement r56	Decay	227.60	39.34	5.924	8.069	1.434	5.47
Joint full map	AUF	307.78	88.43	8.011	10.869	163.840	5.67
Joint full map	Decay	308.23	88.70	8.022	10.933	163.840	5.72

Table 9: All 14 matched architecture/objective cells on NanoCoder. Gain is percent over native DFlash. AR is the mean-throughput ratio to the matched autoregressive control. Fitting time uses two GPUs and excludes cache preparation.

Architecture	Loss	Tok/s	Gain	AR	Accepted	Params M	Fit min
Five full maps	AUF	162.32	8.07	4.161	5.549	32.768	2.76
Full ΔF	AUF	162.45	8.15	4.165	5.542	32.768	2.76
Body LoRA r128	AUF	164.96	9.82	4.229	5.593	36.700	2.81
Replacement r28	AUF	56.40	-62.45	1.446	1.833	0.717	2.61
Replacement r56	AUF	76.12	-49.32	1.951	2.400	1.434	2.54
Replacement r128	AUF	106.96	-28.79	2.742	3.387	3.277	2.59
Body LoRA r128	Decay	163.20	8.65	4.184	5.457	36.700	2.79
Full ΔF	Decay	159.43	6.14	4.087	5.384	32.768	2.70
Five full maps	Decay	154.88	3.11	3.971	5.362	32.768	2.79
Replacement r128	Decay	108.00	-28.10	2.769	3.302	3.277	2.53
Replacement r28	Decay	56.77	-62.20	1.455	1.818	0.717	2.52
Replacement r56	Decay	73.03	-51.38	1.872	2.346	1.434	2.52
Joint full map	AUF	139.62	-7.05	3.579	4.486	163.840	3.46
Joint full map	Decay	137.17	-8.68	3.516	4.465	163.840	3.47

Dense reconstruction control. Using every prompt and response position in the same-model reconstruction control changes throughput versus native by -1.83% on GSM8K, $+20.50\%$ on KiCad, and -1.37% on NanoCoder. Fitting takes 34.12, 381.61, and 37.45 seconds on two GPUs. The corresponding accepted lengths are 4.4696, 7.2064, and 4.9742. Increasing from half to full coverage barely changes these values. This supports the choice of token-based adaptation in the tested specialized setting, subject to differences in feature capture and optimization between recipes.

C COMPLETE SUPERVISION-BUDGET AND COVERAGE RECORDS

Each domain has eight distinct budget settings for each architecture/objective combination. At fixed 4,096 examples, requested anchors are 8, 24, 32, 64, and 128. At fixed 32 anchors, example presentations are 4,096, 8,192, 12,288, and 16,384. Their shared cell appears once. Each fit starts independently and runs one epoch over its constructed list. The update counts are 512, 1,024, 1,536, and 2,048 when increasing presentations. This is two intersecting one-dimensional sweeps, not a full factorial experiment.

The supplied scaling record does not provide accepted-length measurements for the decaying-CE rows. Consequently, the complete tables omit an acceptance column rather than fill it from another experiment. The plotting code normalizes the recorded architecture aliases `maps/maps_full` and `body/body_r128` before grouping, so all 96 points are retained. The intervals below quantify paired prompt variability, not independent hardware repeats or training-seed variability.

Table 10: All 32 budget cells on GSM8K. N counts example presentations and A requested anchors. Gain is percent mean-throughput increase over native. The 95% interval resamples paired prompts within a run. It is not a multi-seed or hardware-repeat interval.

Arch.	Loss	N	A	Gain	95% interval	Fit min
Body	AUF	4096	8	37.88	[34.5, 41.4]	1.32
Body	AUF	4096	24	40.16	[36.7, 43.7]	2.17
Body	AUF	4096	32	36.41	[32.8, 40.1]	2.72
Body	AUF	4096	64	40.14	[36.3, 44.1]	5.14
Body	AUF	4096	128	40.57	[36.7, 44.5]	9.70
Body	AUF	8192	32	41.19	[37.2, 45.1]	5.39
Body	AUF	12288	32	44.96	[41.1, 49.0]	8.10
Body	AUF	16384	32	49.16	[45.0, 53.7]	10.81
Body	Decay	4096	8	30.75	[27.9, 33.8]	1.33
Body	Decay	4096	24	33.74	[30.4, 37.1]	2.17
Body	Decay	4096	32	34.82	[31.6, 38.1]	2.71
Body	Decay	4096	64	34.92	[31.6, 38.4]	5.14
Body	Decay	4096	128	33.10	[29.5, 36.8]	9.66
Body	Decay	8192	32	34.85	[31.4, 38.6]	5.41
Body	Decay	12288	32	39.15	[35.5, 43.2]	8.12
Body	Decay	16384	32	41.05	[37.0, 45.3]	10.85
Maps	AUF	4096	8	34.85	[31.2, 38.5]	1.17
Maps	AUF	4096	24	37.13	[33.6, 40.6]	2.02
Maps	AUF	4096	32	37.58	[34.0, 41.2]	2.66
Maps	AUF	4096	64	38.00	[34.3, 41.8]	4.89
Maps	AUF	4096	128	43.47	[39.6, 47.7]	9.15
Maps	AUF	8192	32	40.88	[36.8, 45.3]	5.16
Maps	AUF	12288	32	46.98	[42.4, 51.6]	7.78
Maps	AUF	16384	32	44.95	[40.7, 49.5]	10.31
Maps	Decay	4096	8	35.28	[32.0, 38.7]	1.11
Maps	Decay	4096	24	32.60	[29.2, 36.0]	2.02
Maps	Decay	4096	32	36.84	[33.3, 40.6]	2.67
Maps	Decay	4096	64	37.41	[33.9, 41.1]	4.86
Maps	Decay	4096	128	36.80	[33.4, 40.4]	9.08
Maps	Decay	8192	32	38.56	[34.9, 42.2]	5.13
Maps	Decay	12288	32	41.68	[37.8, 45.6]	7.68
Maps	Decay	16384	32	44.44	[40.5, 48.7]	10.26

Table 11: All 32 budget cells on KiCad. N counts example presentations and A requested anchors. Gain is percent mean-throughput increase over native. The 95% interval resamples paired prompts within a run. It is not a multi-seed or hardware-repeat interval.

Arch.	Loss	N	A	Gain	95% interval	Fit min
Body	AUF	4096	8	59.33	[55.7, 62.5]	3.12
Body	AUF	4096	24	73.77	[69.3, 77.7]	4.14
Body	AUF	4096	32	75.46	[71.0, 79.5]	5.17
Body	AUF	4096	64	86.51	[81.4, 91.4]	7.50
Body	AUF	4096	128	91.50	[85.8, 96.7]	13.50
Body	AUF	8192	32	94.07	[88.3, 99.6]	10.87

Continued on next page

Table 11 continued

Arch.	Loss	N	A	Gain	95% interval	Fit min
Body	AUF	12288	32	103.20	[96.6, 109.3]	17.71
Body	AUF	16384	32	113.11	[106.3, 119.8]	23.35
Body	Decay	4096	8	67.66	[63.4, 71.4]	3.30
Body	Decay	4096	24	80.95	[76.2, 85.5]	4.34
Body	Decay	4096	32	84.95	[79.6, 89.9]	5.63
Body	Decay	4096	64	91.73	[86.1, 97.1]	7.60
Body	Decay	4096	128	98.95	[92.8, 105.0]	13.70
Body	Decay	8192	32	101.34	[95.1, 107.5]	10.99
Body	Decay	12288	32	109.36	[102.4, 116.0]	17.10
Body	Decay	16384	32	115.97	[108.7, 123.2]	23.46
Maps	AUF	4096	8	59.65	[55.7, 63.2]	2.97
Maps	AUF	4096	24	82.73	[77.3, 87.7]	4.34
Maps	AUF	4096	32	89.52	[83.5, 95.0]	5.32
Maps	AUF	4096	64	94.63	[88.6, 100.6]	7.48
Maps	AUF	4096	128	102.24	[95.6, 108.4]	13.14
Maps	AUF	8192	32	103.60	[97.1, 109.7]	11.04
Maps	AUF	12288	32	112.90	[105.8, 119.7]	17.24
Maps	AUF	16384	32	112.96	[105.7, 119.9]	23.78
Maps	Decay	4096	8	68.77	[64.6, 72.6]	3.51
Maps	Decay	4096	24	95.66	[89.9, 101.0]	4.28
Maps	Decay	4096	32	93.53	[87.6, 99.0]	6.44
Maps	Decay	4096	64	103.96	[97.0, 110.5]	7.38
Maps	Decay	4096	128	107.91	[101.3, 114.2]	12.96
Maps	Decay	8192	32	112.42	[105.6, 119.0]	10.64
Maps	Decay	12288	32	105.50	[99.0, 111.8]	17.31
Maps	Decay	16384	32	110.64	[103.4, 117.5]	23.38

Table 12: All 32 budget cells on NanoCoder. N counts example presentations and A requested anchors. Gain is percent mean-throughput increase over native. The 95% interval resamples paired prompts within a run. It is not a multi-seed or hardware-repeat interval.

Arch.	Loss	N	A	Gain	95% interval	Fit min
Body	AUF	4096	8	9.15	[7.1, 11.1]	1.37
Body	AUF	4096	24	11.33	[9.1, 13.6]	2.24
Body	AUF	4096	32	9.82	[7.7, 11.9]	2.81
Body	AUF	4096	64	12.29	[10.2, 14.4]	5.21
Body	AUF	4096	128	10.39	[8.2, 12.7]	10.34
Body	AUF	8192	32	14.79	[12.6, 17.0]	5.65
Body	AUF	12288	32	15.55	[13.1, 18.2]	8.49
Body	AUF	16384	32	11.55	[9.2, 14.2]	11.31
Body	Decay	4096	8	8.72	[6.8, 10.8]	1.38
Body	Decay	4096	24	9.16	[7.0, 11.5]	2.23
Body	Decay	4096	32	8.65	[6.5, 11.1]	2.79
Body	Decay	4096	64	10.28	[8.1, 12.5]	5.20
Body	Decay	4096	128	9.23	[7.1, 11.5]	10.31
Body	Decay	8192	32	10.73	[8.6, 13.0]	5.54
Body	Decay	12288	32	12.81	[10.3, 15.6]	8.32
Body	Decay	16384	32	14.37	[12.0, 17.0]	11.16
Maps	AUF	4096	8	4.70	[2.5, 7.0]	1.17
Maps	AUF	4096	24	10.36	[8.2, 12.6]	2.09
Maps	AUF	4096	32	8.07	[5.7, 10.4]	2.76
Maps	AUF	4096	64	12.65	[10.3, 15.0]	4.98
Maps	AUF	4096	128	12.70	[10.3, 15.3]	9.77

Continued on next page

Table 12 continued

Arch.	Loss	N	A	Gain	95% interval	Fit min
Maps	AUF	8192	32	14.67	[12.2, 17.2]	5.46
Maps	AUF	12288	32	11.40	[8.8, 14.2]	8.17
Maps	AUF	16384	32	13.10	[10.2, 16.0]	10.94
Maps	Decay	4096	8	5.15	[2.9, 7.5]	1.19
Maps	Decay	4096	24	8.68	[6.6, 10.8]	2.10
Maps	Decay	4096	32	3.11	[0.8, 5.5]	2.79
Maps	Decay	4096	64	11.31	[8.8, 13.9]	4.94
Maps	Decay	4096	128	9.01	[6.7, 11.4]	9.73
Maps	Decay	8192	32	10.49	[8.1, 13.0]	5.29
Maps	Decay	12288	32	10.90	[8.3, 13.7]	7.99
Maps	Decay	16384	32	10.43	[8.0, 13.1]	10.70

Table 13: All eight completed T1 DIR example/position-coverage recipes. Workload entries give native-relative mean request throughput. Coverage includes prompt and response positions.

Examples	Coverage	Fit min	MATH	GSM8K	Code	Chat
16384	25%	24.91	1.0022	0.9940	0.8416	0.9043
16384	50%	37.21	0.9989	0.9848	0.8477	0.9053
8192	100%	53.90	0.9979	0.9901	0.8283	0.8914
8192	50%	17.79	0.9986	0.9753	0.8322	0.8882
8192	25%	11.59	0.9953	0.9686	0.8309	0.8749
4096	100%	18.30	0.9971	0.9797	0.7785	0.8537
4096	25%	6.06	0.9974	0.9756	0.7623	0.8406
4096	50%	9.56	0.9764	0.9422	0.7522	0.8096

D CROSS-MODEL BASELINES AND SERVING RECORDS

Table 14: Complete standalone autoregressive controls and selected DIR results. Every row uses 128 prompts, one L40S, and a 2,048-token response cap. Rates are arithmetic mean request tokens/s.

Pair	Workload	AR tokens/s	Mapped tokens/s	Mapped / AR
T1	MATH	37.8373	220.26	5.8212
T1	GSM8K	37.7622	176.87	4.6838
T1	Code	37.4970	127.61	3.4032
T1	Chat	35.0880	80.93	2.3065
T2	MATH	39.4153	216.37	5.4895
T2	GSM8K	39.7241	169.53	4.2677
T2	Code	39.6034	130.83	3.3035
T2	Chat	40.4786	81.16	2.0050
T3	MATH	61.8956	145.43	2.3496
T3	GSM8K	64.4510	140.60	2.1815
T3	Code	63.0876	118.57	1.8794
T3	Chat	64.7250	104.43	1.6134
T4	MATH	41.0048	121.94	2.9738
T4	GSM8K	41.1094	90.84	2.2097
T4	Code	40.7880	80.60	1.9761
T4	Chat	40.9593	60.52	1.4776

For T1, native standalone mean rates on MATH/GSM8K/Code/Chat are 219.79/177.94/151.63/89.50 tokens/s. T2 rates are 224.52/177.32/171.68/95.34. T4 rates are 130.07/127.35/122.44/114.42. Native-drafter recovery divides each mapped rate by that matched native rate. It is not the fraction of the native drafter’s improvement over AR. No native T3 rate is supplied.

Table 15: Selected transfer accepted lengths. Values use each implementation’s recorded convention including the verifier contribution and terminal truncation. T3 lacks a native comparator and is reported separately in text.

Pair	Drafter	MATH	GSM8K	Code	Chat
T1	Native	8.211	6.479	5.644	3.324
T1	Mapped	7.984	6.242	4.739	2.887
T2	Native	8.091	6.342	6.262	3.361
T2	Mapped	7.660	5.987	4.889	2.829
T4	Native	5.121	4.425	4.523	3.954
T4	Mapped	4.634	3.226	3.054	2.064

T3 mapped accepted lengths are 4.780, 4.159, 3.440, and 2.974 across the same four workloads. The source of the vocabulary head, head cost, draft depth, and tokenizer bridge differ across transfer pairs. Lower accepted length can coexist with comparable throughput when the retained draft body is less expensive.

D.1 SPECIALIZED SHARED-SERVING RECORDS

Table 16: All recorded five-map Qwen3-4B serving configurations. Mean request and aggregate throughput have different denominators. Explicit 4B twelve-client and learned-router rows use capacity 12. Other explicit-ID rows use capacity 32.

Selection	Clients	Method	Mean tokens/s	Aggregate tokens/s
explicit	1	AR	64.39	63.93
explicit	1	DFlash	270.21	291.61
explicit	1	EAGLE-3	135.14	136.27
explicit	1	RelaySpec	390.71	516.24
explicit	4	AR	55.44	226.28
explicit	4	DFlash	192.75	889.11
explicit	4	EAGLE-3	111.32	484.34
explicit	4	RelaySpec	283.22	1572.09
explicit	8	AR	49.44	390.70
explicit	8	DFlash	171.82	1534.73
explicit	8	EAGLE-3	92.52	775.09
explicit	8	RelaySpec	246.99	2689.53
explicit	12	AR	43.81	491.38
explicit	12	DFlash	138.99	1842.77
explicit	12	EAGLE-3	78.53	967.81
explicit	12	RelaySpec	202.42	3227.26
explicit	16	AR	40.91	595.46
explicit	16	DFlash	124.67	2097.75
explicit	16	EAGLE-3	73.87	1150.17
explicit	16	RelaySpec	175.93	3693.09
explicit	32	AR	31.01	827.73
explicit	32	DFlash	75.22	2439.74
explicit	32	EAGLE-3	58.31	1639.70
explicit	32	RelaySpec	108.14	4323.96
learned	12	AR	43.80	503.17
learned	12	DFlash	140.44	1824.21
learned	12	EAGLE-3	78.71	960.25
learned	12	RelaySpec	201.98	3195.90

Table 17: All recorded five-map Qwen3-8B serving configurations. Mean request and aggregate throughput have different denominators. Explicit 4B twelve-client and learned-router rows use capacity 12. Other explicit-ID rows use capacity 32.

Selection	Clients	Method	Mean tokens/s	Aggregate tokens/s
explicit	1	AR	39.22	39.69
explicit	1	DFlash	156.97	159.17
explicit	1	EAGLE-3	77.87	80.67
explicit	1	RelaySpec	183.69	178.59
explicit	4	AR	35.38	143.12
explicit	4	DFlash	130.31	525.87
explicit	4	EAGLE-3	72.74	299.96
explicit	4	RelaySpec	150.72	593.43
explicit	8	AR	35.20	272.58
explicit	8	DFlash	120.15	931.62
explicit	8	EAGLE-3	68.61	555.52
explicit	8	RelaySpec	139.09	1043.65
explicit	16	AR	33.51	495.40
explicit	16	DFlash	97.12	1482.33
explicit	16	EAGLE-3	62.56	945.01
explicit	16	RelaySpec	110.93	1431.21
explicit	32	AR	30.59	512.06
explicit	32	DFlash	61.98	1569.56
explicit	32	EAGLE-3	55.80	1270.93
explicit	32	RelaySpec	70.05	1521.07
learned	12	AR	34.20	394.54
learned	12	DFlash	108.00	1248.05
learned	12	EAGLE-3	63.43	737.91
learned	12	RelaySpec	124.28	1384.27

The learned router uses TF-IDF features and a logistic classifier. It predicts domain labels, not response correctness. Its accuracy is 383/384 at 4B and 255/256 at 8B. Fitting uses 12,288 and 8,192 labeled prompts and takes 0.889 and 3.233 CPU seconds. Every method in a comparison receives the same predicted target assignments. This prevents a target-selection change from being credited as a draft improvement.

The earlier rank-56 residual-fusion experiment is kept separate from the five-map results. It uses $F_e = F + B_e A_e$, 4,096 examples per domain, eight anchors, and 2,000 AUF updates. At twelve clients, explicit selection reaches 1.3176 and 1.3288 times native aggregate throughput in two order-reversed repeats. Learned selection reaches 1.3212 and 1.3243. One pooled residual reaches 98.94–99.60% of the separate-bank explicit throughput and 99.42–99.58% under learned selection. This control limits claims that draft specialization must always be private to each target.

D.2 CROSS-MODEL CONCURRENT SERVING

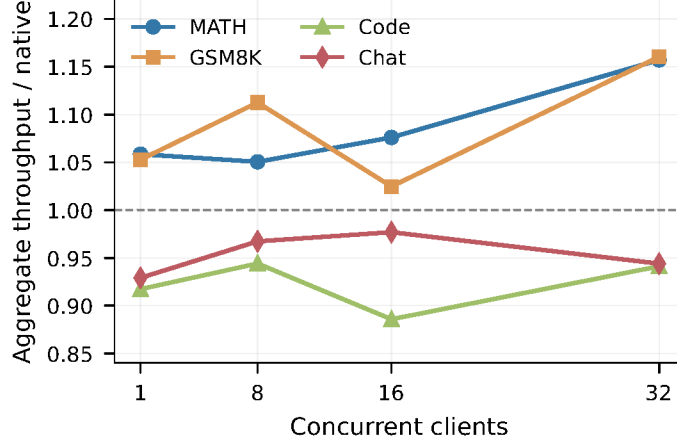


Figure 8: T1’s selected cross-model DIR fit under optimized vLLM. Each native or mapped point contains 128 requests. Native-relative aggregate speed is workload-dependent. Lines connect four measured client settings and are not extrapolations.

Table 18: Cross-model serving with measured native controls. T1 has clients 1/8/16/32 and T2/T4 have 16. Each method/workload/client cell uses 128 requests. All rates are aggregate tokens per workload wall second, including drain.

Pair	Task	Clients	Native	Mapped	Ratio	Native τ	Map τ
T1	chat	1	114.83	106.68	0.9291	3.325	2.851
T1	chat	8	705.51	682.44	0.9673	3.305	2.908
T1	chat	16	1091.14	1065.93	0.9769	3.312	2.898
T1	chat	32	1349.44	1273.82	0.9440	3.355	2.917
T1	code	1	192.43	176.52	0.9173	5.719	4.826
T1	code	8	1106.87	1045.05	0.9441	5.716	4.893
T1	code	16	1748.19	1548.47	0.8858	5.906	4.893
T1	code	32	1940.52	1826.45	0.9412	5.680	4.843
T1	gsm	1	216.54	227.95	1.0527	6.454	6.248
T1	gsm	8	1301.71	1448.26	1.1126	6.436	6.229
T1	gsm	16	2207.43	2262.03	1.0247	6.450	6.254
T1	gsm	32	2251.47	2612.56	1.1604	6.458	6.256
T1	math	1	276.09	292.23	1.0585	8.102	7.917
T1	math	8	1752.39	1840.94	1.0505	8.138	7.942
T1	math	16	2524.31	2716.26	1.0760	8.110	7.930
T1	math	32	2866.96	3317.43	1.1571	8.020	7.827
T2	chat	16	1888.69	1431.21	0.7578	3.393	2.788
T2	code	16	2840.15	2002.79	0.7052	6.163	4.537
T2	gsm	16	3160.99	3078.87	0.9740	6.366	6.013
T2	math	16	4482.42	3710.76	0.8278	8.109	7.534
T4	chat	16	1440.04	524.40	0.3642	3.986	2.043
T4	code	16	1296.83	636.97	0.4912	4.387	2.987
T4	gsm	16	1354.50	853.13	0.6299	4.211	3.439
T4	math	16	1895.00	1046.94	0.5525	5.091	4.681

Table 19: All available cross-model concurrent autoregressive comparisons. These twelve rows are T2–T4 at sixteen clients. The T1 serving AR baseline is unmeasured.

Pair	Task	Clients	AR	Mapped	Mapped / AR	Map τ
T2	chat	16	941.99	1431.21	1.5193	2.788
T2	code	16	825.29	2002.79	2.4268	4.537
T2	gsm	16	1021.54	3078.87	3.0139	6.013
T2	math	16	959.42	3710.76	3.8677	7.534
T3	chat	16	1072.82	1899.95	1.7710	2.989
T3	code	16	1213.67	2077.62	1.7119	3.529
T3	gsm	16	727.30	2241.59	3.0821	3.853
T3	math	16	1126.93	2856.09	2.5344	4.713
T4	chat	16	520.09	524.40	1.0083	2.043
T4	code	16	585.58	636.97	1.0878	2.987
T4	gsm	16	394.44	853.13	2.1629	3.439
T4	math	16	558.22	1046.94	1.8755	4.681

Aggregate rate divides returned tokens by workload completion time including drain. Request latency starts after acquisition of a client slot. It is not a measure of waiting time in an unlimited external arrival queue. TTFT and TPOT are per-request means. Memory snapshots in the supplied records are allocated or reserved worker memory after the cell, not peaks. For example, T1’s native allocated snapshot is 30.5914 GiB and the mapped snapshot is 30.9190 GiB. These data do not support a universal inference-memory reduction claim.

E ADDITIONAL EVIDENCE FOR THE OBJECTIVE CHOICE

E.1 CROSS-MODEL RECIPE AND CAPACITY CONTROLS

Table 20: T1 architecture/recipe controls using 8,192 examples and three epochs. These are not the main 16k/25% fit. Body controls adapt the larger native 8B draft body, not the transferred 4B body. Every entry is mean request tokens/s.

Recipe	MATH	GSM8K	Code	Chat	Fit min
Five-map DIR50	219.48	173.55	126.18	79.49	17.79
Five-map cold AUF	194.22	162.05	58.37	49.93	31.64
Five-map decay CE	194.69	163.47	61.56	51.31	28.24
Native body AUF	230.82	194.86	155.33	91.96	31.79
Native body decay CE	226.15	187.66	153.86	90.26	31.78

The native-body comparator uses rank-128 updates with 48.497M trainable parameters. It gives the strongest selected rates in this table. Its different pretrained body means it is not an isolated comparison of two ways to fit the same transferred model. Peak fitting memory is 3.89 GiB for DIR50, 5.53 GiB for AUF, 5.52 GiB for decay, and 8.58 GiB for native-body training. These are training-loop measurements, not peak serving memory.

E.2 GEOMETRY, COMMON-INPUT DIAGNOSTICS, AND INTERPOLATION

The training-feature geometry probe uses 64 records and 7,090 positions, split into 2,994 prompt positions and 4,096 response positions. It pools positions, unlike the example-weighted fitting loss. Geometry probes use $\max(\|v\|^2, 10^{-6})$ in the denominator rather than the additive epsilon in Equation 5. DIR response-layer cosines with the reference range from 0.944 to 0.985, while cold AUF ranges from 0.121 to 0.267. These values measure representation direction, not speed.

The common-input diagnostic selects eight prompt-length quantiles per task from the existing evaluation cohort. Eight fixed anchor blocks per example give 64 blocks per task. Every map sees identical saved prefixes and anchors. It is a small diagnostic subset, not an independent final test.

Figure 4 reports cross-entropy over all valid labels regardless of training loss. Live interpolation then evaluates 128 prompts, retaining the fixed draft body. The midpoint Code rates are 81.64 tokens/s from AUF and 84.68 from decay, compared with parent rates 58.37 and 61.56. The DIR endpoint reaches 126.18. Summed request latency falls by 27.53% and 26.39% relative to the respective token parent. Endpoints come from separate timing passes.

Table 21: Warm AUF from the 8k/DIR50 parent. Workload entries are mean-throughput ratios to native DFlash. Seen/unseen refer to overlap with the parent’s training examples. Fitting minutes include both stages.

Refinement	MATH	GSM8K	Code	Chat	Total fit min
Seen 4k	0.9717	1.0339	0.7273	0.8420	31.52
Seen 8k	0.9927	1.0431	0.7043	0.8446	45.32
Unseen 4k	0.9694	1.0413	0.7417	0.8548	32.39
Unseen 8k	0.9911	1.0606	0.7051	0.8431	47.42

GSM8K improves in each completed refinement, whereas the other workloads decrease relative to DIR. This does not by itself prove overfitting. Establishing that would require matched training and held-out learning curves. The later broader continuation campaign was stopped and is not treated as a completed factorial study.

Table 22: Rectangular-identity retry on T1. All maps start at $[I_{2560} \ 0]$ and all learning rates are 10^{-3} . DIR uses 25% coverage and token objectives use 32 anchors. Both initialization and learning rate change relative to the original cold token fits.

Recipe	MATH TPS	GSM8K TPS	Code TPS	Chat TPS	Fit min
DIR	213.70	166.70	119.49	76.13	11.11
AUF	206.16	175.35	73.10	60.77	30.14
Decay CE	203.74	169.72	76.02	61.11	29.54

AUF is faster than DIR on GSM8K in this retry, despite lower accepted length. That is another reason not to infer wall-time rankings from a proposal statistic. Code and Chat still strongly favor DIR. The main selected cross-model recipe remains Xavier initialization, not this diagnostic retry.

E.3 DESTRUCTIVE INTERVENTIONS ON A CROSS-MODEL AUF PARENT

These interventions act on the completed T1 AUF parent, not DIR. Let $W_{0,i}$ be its random initialization and $\Delta W_i = W_i - W_{0,i}$. Resetting the last map to $W_{0,i}$ reduces MATH throughput to 0.3643 of the parent. Resetting the first map retains 0.9626. Removing 32 fusion-sensitive input directions retains only 0.1978 of parent throughput, while removing 32 random orthonormal directions retains 0.9983. Keeping rank two of each ΔW_i above the random initial map gives accepted length 1.078. Rank 128 retains 0.7902 of parent throughput. These tests show functional use of the learned interface and reject a universal rank-two explanation. The corresponding Code removal cohorts were stopped before completion and are not reported as full results.

E.4 SPECIALIZATION GEOMETRY AND COMPRESSION

Whole-map truncation replaces W_i by its rank- r approximation. Identity-preserving truncation replaces it by $I + [W_i - I]_r$. If W_i were exactly I , the first operation would still delete most input directions. Therefore whole-map failure cannot establish the intrinsic rank of the learned change. For math, rank-128 changes retain 95.1% and 99.6% of the full mapper’s summed-time reduction in two timing rounds. NanoCoder nearly preserves accepted length at high retained ranks but shows substantially noisier time-saving retention. Every static intervention is folded into a dense projection. No low-rank runtime kernel is benchmarked.

A radial/perpendicular split tests a different object, the actual correction δ at a context position. With $s = \sqrt{\|z\|^2/d_d + \epsilon}$, the RMSNorm derivative is

$$J_N(z) = \text{diag}(g) \left(I/s - zz^\top / (d_d s^3) \right). \quad (9)$$

The normalization epsilon is 10^{-6} , distinct from the 10^{-20} projection safeguard. Positive radial scaling disappears exactly only in the zero-epsilon limit. Finite corrections, sign reversal, and downstream nonlinear computation limit broader claims. Relative errors of the first-order RMSNorm prediction are 17.55% on math and 14.88% on NanoCoder. Radial energy fractions are 25.22% and 34.82%, despite little radial-only accepted-length benefit. Energy percentages are not speedup fractions.

To test literal reversal of the target adapter, replay the same saved tokens through the base and adapted targets. Let b be the base fused context, z_0 the adapted-target context without a mapper, and $t = z_0 - b$. Exact restoration before normalization requires the mapper correction $\delta = -t$. Mean positionwise cosines over 1,024 sampled training response positions are -0.0429 on math and $+0.1248$ on NanoCoder. Shuffled-position controls give -0.0413 and $+0.1052$. These averages do not support systematic position-specific cancellation, but also do not rule out heterogeneous effects or useful recovery after normalization.

Why the parameterization matters. Let $G_i = F_i W_i$ and let $H_i = \partial \mathcal{L} / \partial G_i$ be the effective fusion gradient. Plain gradient descent with step size η gives

$$\frac{\partial \mathcal{L}}{\partial W_i} = F_i^\top H_i, \quad \Delta G_i = -\eta F_i F_i^\top H_i. \quad (10)$$

Direct fusion updates instead use $-\eta H_i$. Even when the two parameterizations can represent the same effective matrix, they induce different updates. This identity does not transfer unchanged to AdamW. Actual zero-moment Adam diagnostics give effective-update cosines 0.238–0.515 on math and 0.269–0.495 on NanoCoder. They demonstrate different update directions, not a proof of which final fit will win.

F LITERATURE COMPARISON AND PLANNED EXTENSIONS

The EAGLE series, Medusa, ParallelSpec, and PaSS improve proposal generation in different ways (Li et al., 2024b,c, 2025, Cai et al., 2024, Xiao et al., 2024, Monea et al., 2023). SpecInfer changes tree-based serving, while SSD overlaps drafting and verification (Miao et al., 2023, Kumar et al., 2026). These mechanisms are not experimentally combined with our maps. LayerSkip changes target training to enable early exits (Elhoushi et al., 2024). DFlare increases draft capacity and conditioning complexity (Zhang et al., 2026). Our study deliberately fixes the trained draft body. Model stitching and representation finetuning provide broader precedents for adapting internal interfaces (Bansal et al., 2021, Wu et al., 2024). Representation similarity alone does not establish downstream utility (Kornblith et al., 2019), which is why the live interventions matter.

F.1 PUBLISHED SPEEDUPS ARE NOT MATCHED MEASUREMENTS

Table 23: Published Qwen3-8B context from the attached reference papers. Different hardware, workloads, and engines prevent a numerical ranking against RelaySpec. These are literature values, not reproduced experiments.

Paper / mode	Measurement setup	Tasks averaged	AR speedup
DFlash	Transformers on H200, nonthinking, temperature zero	7	4.86×
PARD-2 dependent	vLLM on A100-40GB, nonthinking, temperature zero	4	5.81×
PARD-2 independent	Same PARD-2 evaluation configuration	4	4.82×

DFlash’s architecture and PARD-2’s deployment question are both relevant, but they do not establish a common speed denominator (Chen et al., 2026, An et al., 2026b). DFlash’s newer serving results use SGLang (Zheng et al., 2024), whereas this work’s serving experiments use vLLM. The papers also train different draft bodies with different data. No factor between their published ratios and ours is interpreted as a measured method improvement.

PARD-2’s confidence weighting uses target confidence along the prefix. AUF uses the current drafter’s discrete prediction outcomes to choose a detached training support. Neither is a new objective of RelaySpec. PARD-2’s optional conditioning and target-independent operation are genuine advantages when no per-target fitting is desired. RelaySpec instead serves settings where an existing feature-conditioned body is available and a modest offline interface fit is acceptable. The supplied records support this alternative, not a claim that one design dominates every deployment.

F.2 HARDWARE COMPARISON PROTOCOL, RESULTS RESERVED

The hardware study should keep model and adapter revisions, tokenization, prompts, precision, stopping, and draft budget fixed. Report the exact A100 memory size and form factor and the DGX Spark software stack. Compare single-request latency and saturated aggregate throughput separately. Use the same request cohorts on L40S, A100, and DGX Spark. Separate compilation and warmup from timed generation. Repeat timing with reversed method order and record variability. Do not combine one device’s AR baseline with another device’s speculative result. A hardware-specific kernel optimization should be separately labeled rather than presented as the effect of the maps.

A100 / DGX Spark results insertion. Place the completed hardware table and throughput/latency figure here. Retain a column only when all displayed method rows have that measurement.

F.3 MEASURED FLOATING-POINT OPERATIONS AND MEMORY, RESULTS RESERVED

At decoding round r , record draft operations $f_{D,r}$, target verification operations $f_{V,r}$, and any separate runtime overhead. Let n_r be the number of tokens actually emitted in that round. A workload-level operations-per-emitted-token estimate is

$$f_{\text{emitted}} = \frac{\sum_r (f_{D,r} + f_{V,r} + f_{\text{other},r})}{\sum_r n_r}. \quad (11)$$

Report how each count is obtained, whether multiply-add is counted as two operations, and which attention, vocabulary-head, and bridge operations are covered. Divide the corresponding AR operation count by its emitted-token count before comparison. The fraction of retained proposals is not automatically the fraction of useful hardware FLOPs. Target verification, temporary attention, and boundary effects can scale differently.

For memory, report peak allocated and reserved bytes after resetting profiler peaks, not only a post-run snapshot. Separate target weights/KV, frozen draft weights/KV, mapped fusion banks, temporary activations, and tokenizer/host storage. Exporting one projection removes a serial operation but not these other costs. Show time per draft, time per verify, prefill time, and end-to-end latency alongside FLOPs.

FLOPs and overhead results insertion. Insert measured component timings, operations per emitted token, and peak-memory breakdown. Section 8’s analytic counts remain labeled as analytic.

F.4 ULTRA-LONG CONTEXT PROTOCOL, RESULTS RESERVED

Use increasing input-context lengths up to the actual supported limit of the pinned model. Report token counts after each tokenizer’s encoding. Keep output stopping natural and its cap fixed, and distinguish long inputs from long generated answers. LongBench tests real tasks and RULER supplies controlled long-context difficulty (Bai et al., 2023, Hsieh et al., 2024). Compare untouched native and mapped checkpoints first. Any additional long-context training, such as a LongAlign-based procedure, should then be reported as a separate model with its own fitting cost (Bai et al., 2024). Context-level failures, truncation rates, per-round acceptance, prefill latency, decode latency, and memory should all be retained. No ultra-long-context result is part of the present evidence.

Ultra-long context results insertion. Add context-length curves for throughput, accepted length, prefill time, and peak memory. Do not extrapolate short-context curves to unmeasured context lengths.